Model-based Coverage Motion Planning for Industrial 3D Shape Inspection Applications（IEEE CASE 2017, pp. 1293–1300）

复现方案

Zhouyiyang Yang

Table of contents

1 目标+范围 2
2 1. 论文管线模块化视图 2
3 环境搭建 3
4 项目结构 3
5 M0 工件准备（Day 1） 4
6 M1 表面离散化（Day 1 – 2） 4
7 M2 视点采样（Day 3 – 4） 5
7.1 位置采样 5
7.2 视线方向：势场法（Eq.10） 5
7.3 可达性过滤（替代论文的 IK+FCL，简化偏差 #2） 6
7.4 自检 6
8 M3 可见性矩阵（Day 5 – 7，工作量主体） 6
8.1 四条判据（对每个面片的全部 3 个顶点逐一判定，全过才算可见） 6
8.2 矩阵组装 7
8.3 覆盖率函数（fitness 与验证共用） 7
8.4 自检（诊断图） 7
8.5 手算单元测试（6 个，写在 tests/test_visibility.py） 7
9 M4 行走代价 + home pose（Day 8） 8
10 M5 RKGA 求解 SCTSP（Day 9 – 11） 8
10.1 问题表述（论文 Eq.8 – 9, 11 – 13，实现按此） 8
10.2 染色体编码与解码 8
10.3 fitness（论文 Algo 1；实现按 Eq.11 的求和形式） 8
10.4 GA 循环（参数为自选——论文未给出，报告时注明） 9
10.5 分阶段调试顺序（重要，别跳步） 9
11 两个 baseline（Day 12 – 13） 10
11.1 Greedy-SCP-GA-TSP 10
11.2 GA-SCP-GA-TSP 10
12 评测 harness（Day 14 – 15） 10
12.1 输出表格（对照论文 Table II/III 格式） 10
12.2 收敛曲线（对照论文 Fig.10） 10
12.3 耗时记录 10
13 复现正确性自检 checklist（分三级） 11
14 Phase 2 衔接：rollout 数据生成（Day 16 – 18） 11
14.1 轨迹插值（EE 空间） 11
14.2 渲染图像 11
14.3 覆盖向量时序（Option B/C 的条件信号） 11

14.4 位姿扰动版本（Phase 3 R1 需要） 11
14.5 输出格式 11
15 参数总表（configs/base.yaml 内容） 12
16 时间线（共 2.5 周 + 3 天缓冲） 12
17 完成后直接可用的三样东西 13

1 目标+范围

要做的事：完整复现论文的离线管线——CAD 网格 → 视点采样（势场定视线）→ 可见性矩阵 → 行走代价 → RKGA 单次求解 SCTSP → 有序视点序列 + cycle time；并同时实现两个分开求解的 baseline（GA-SCP-GA-TSP、Greedy-SCP-GA-TSP）。

验收标准：

1. 3 个工件上，几何管线能达到 ≥95% 覆盖（论文 δ=99.5%，我们的验收线放宽到 95%，但实现按 δ=99.5% 跑）；
2. **相对模式复现**：GA-SCTSP 的 cycle time < GA-SCP-GA-TSP，改善幅度落在 5–25% 区间；且 GA-SCP-GA-TSP 的视点数**最少**（"点少≠时间短"现象）；
3. RKGA 在 100–150 代内收敛，10 trials 的 T_std 稳定；
4. 单个 (工件, 400 候选) 实例的完整求解 ≤ 30 分钟（我们的规模比论文小，travel cost 用代理，应该远低于此）；
5. 产出 ≥50 条 rollout 演示轨迹（含 ≥10 条位姿扰动版本），供 Phase 2 使用。

明确不做的（简化偏差，与论文的 6 处差异）：

#	论文做法	本复现做法	影响
1	MoveIt + OMPL(RRT-Connect) + cubic spline 计算行走时间	travel-cost 代理 $e = \alpha\|\Delta p\| + \beta\cdot\Delta\theta$（见 §8）	绝对 cycle time 不可与论文直接比较；**相对模式可比**
2	IKFast 求逆解 + FCL 碰撞过滤视点	可达性壳过滤（球形壳 + 桌面高度，见 §6）	不保证机器人可达性；视点合法性偏乐观
3	ABB IRB-4600 真机 + ROS-Industrial	纯仿真	与 Phase 1 定位一致
4	论文未给出 GA 超参数（种群/交叉/变异/代数）	自选参数并在 §9 明确标注	需在论文中报告
5	d_max（Eq.10）与 home pose 数值论文未给出	自选并在 §6/§8 标注	同上
6	工件为论文私有模型	公开模型库替代（§4）	工件不同，绝对数字不同

关键心态：这次复现的"正确性"不是数字相等（工件、机器人、代价函数都不同，绝对数字必然不同），而是**结构现象重现**：联合求解优于分开求解、视点数少不意味着时间短、收敛曲线形态正确。§12 给了可执行的判别清单。

2 1. 论文管线模块化视图

```
┌──────────────────────────────────────────────────────────────┐
│ INPUT                                                        │
```

```
┌──────────────────────────────────────────────────────────────────┐
│  工件 CAD 网格（bbox ~0.24×0.47×0.30 m）                          │
│  传感器参数（Artec Eva）：FOV 30°×21°、FOD 0.40–1.00 m、          │
│    视角 ≤75°、像素 536×371                                        │
│  覆盖率要求 δ = 99.5%   ·   单视点检测时间 ω₀ = 1.0 s             │
│  候选视点数 n ∈ {200, 300, 400}                                   │
└──────────────────────────────┬───────────────────────────────────┘
                               ▼
[M1 表面离散化]  mesh ──► S = {s_1..s_m}（m 个面片，每片带顶点+法向）
                               ▼
[M2 视点采样]  S + 传感器参数 ──► V = {v_1..v_n}（位置 + 视线方向，Eq.10 势场）
                               ▼
[M3 可见性判定]  V × S ──► A ∈ {0,1}^{n×m}（FOV ∧ FOD ∧ 视角 ∧ 无遮挡）
                               ▼
[M4 行走代价]  V × V ──► G(V,E)，e_ij = 视点间行走时间（+ home pose 往返边）
                               ▼
[M5 联合求解]  A + E + δ ──► RKGA 单次求解 SCTSP（Eq.11）
                                ──► 有序视点序列 x' + 总 cycle time
                               ▼
OUTPUT:  视点序列 + cycle time（+ 可选：每个视点的渲染图像、覆盖向量时序）
```

具体例子（沿用之前教学示例工件 W，bbox 300×200×150 mm、1000 面片、200 候选）：

```
M1:  mesh 82,672 面片 → 规整重采样 → S，|S| = 1000
     例: s_347 = 顶点 {(86.8,44.5,60.0),(88.1,45.0,59.4),(87.2,46.3,60.2)},
         法向 (0.12, 0.85, 0.51)，中心 (87.3, 45.1, 60.0) mm
M2:  v_137 = 位置 (180, −120, 240) mm + 视线方向 (−0.61, 0.41, −0.67)
     （方向由 Eq.10：附近面片的 1/d² 加权平均，见 §6）
M3:  A[137][347] = 1  ⇔ 面片 347 的三个顶点全部满足四条件（§7）
M4:  E[137][3] = 0.72 s（行走代理，§8）
M5:  x' = (v_137 → v_3 → v_87 → ...)，n_x = 9 个视点
     总时间 = 9×1.0 s + Σ 行走 ≈ 18.4 s（数量级对照论文真机 19.2 s）
```

3 环境搭建

```bash
# Python 3.11+
mkdir -p ~/work/paper05_replication && cd ~/work/paper05_replication
python3 -m venv .venv && source .venv/bin/activate
pip install numpy scipy matplotlib pyyaml trimesh networkx
# 可选（推荐）：加速 ray casting
pip install rtree    # trimesh 的 ray 查询需要；embree 见 §7 注
# 可选：Phase 2 渲染用（Phase 1 数值部分不需要）
pip install pyrender  # 或使用系统安装的 Blender（paper_08 路线）
```

版本无特殊要求；全部为纯 CPU 计算，无 GPU 依赖。

4 项目结构

```
paper05_replication/
├── configs/base.yaml          # 全部参数（§14 总表）
├── data/
│   ├── meshes/                # 工件网格（obj/stl，§4）
│   └── outputs/               # A.npz, E.npz, results.csv, rollout/
```

```
├── src/
│   ├── m1_discretize.py       # 表面离散化
│   ├── m2_sample.py           # 视点采样（Eq.10 势场）
│   ├── m3_visibility.py       # 可见性矩阵（四判据 + ray casting）
│   ├── m4_travel.py           # 行走代价 + home pose
│   ├── m5_rkga.py             # RKGA + decodeBySort + fitness（Eq.11）
│   ├── baselines.py           # GA-SCP-GA-TSP / Greedy-SCP-GA-TSP
│   ├── evaluate.py            # 10 trials + 表格 + 收敛曲线
│   └── rollout.py             # Phase 2 数据生成（§13）
├── tests/
│   └── test_visibility.py     # §7.5 的 6 个手算用例
└── README.md
```

每个模块一个文件、独立可跑（`python -m src.m3_visibility --config configs/base.yaml`），中间产物落盘（`A.npz/E.npz`）保证断点续跑。

5 M0 工件准备（Day 1）

来源优先级：

1. **调试件（必做）**：用 trimesh 组合图元自制——`trimesh.creation.box` + `cylinder` 布尔差（trimesh 布尔需要 `manifold3d` 后端，`pip install manifold3d`），做一个"带 4 个通孔的铝支架"形状。目的：几何已知，便于手算单测。
2. **论文量级件**：Thingi10K（水密子集）或 ABC 数据集里选 2 个机械零件（齿轮箱、法兰、支架类），`trimesh` 归一化缩放到 bbox 对角 ~0.3–0.5 m（与 paper_05 的 Target Object 1 = 0.24×0.47×0.30 m 同量级，保证与 FOD 0.4–1.0 m 匹配）。
3. **可选**：paper_09 公开的 Gear/Hirth/Crankshaft 模型（该文公开了模型与代码）——选它的理由是与九篇文献共用一个工件，Phase 3 对比时有文献数字可引用。

准备工作（每个网格做一次）：

```python
import trimesh
mesh = trimesh.load("data/meshes/part.obj", force="mesh")
assert mesh.is_watertight or mesh.repair.fix_normals()  # 非水密先修复
mesh = mesh.apply_scale(...)   # 缩放到目标 bbox
mesh.export("data/meshes/part_norm.obj")
```

摆放约定（全文统一坐标系）：工件 bbox 中心在原点，z 轴朝上；桌面平面 z = −(bbox 高度/2) − 0.05 m；"机器人基座"概念上位于 (0.9, 0, 0) m（论文：工件距基座 0.9 m），仅用于 home pose 与可达壳（§6/§8）。

6 M1 表面离散化（Day 1–2）

黑盒

```
输入:  三角网格 mesh（任意面片数，如 82,672）
输出:  面片集 S：m 个面片，每个面片 =（3 个顶点坐标，单位法向 n_j，中心 c_j）
       m 由参数控制（本复现取 1000）
例子:  s_347 = 顶点 {(86.8,44.5,60.0),(88.1,45.0,59.4),(87.2,46.3,60.2)},
       法向（0.12,0.85,0.51）（向外），中心（87.3,45.1,60.0）mm
```

实现要点：

```python
# src/m1_discretize.py
import trimesh, numpy as np

def discretize(mesh: trimesh.Trimesh, m: int = 1000) -> dict:
    # 均匀重采样到约 m 个面片（trimesh 的 remesh 或简化+细分组合）
    # 关键：输出面片尺寸尽量均匀—paper 的可见性按"面片顶点"判定，
    # 面片过大/过小都会让 A 的含义失真。目标边长 ≈ sqrt(总表面积/m)。
    patch = {
        "vertices": np.array(...),   # (m,3,3) 每片 3 顶点
        "normals":  np.array(...),   # (m,3)   向外法向（trimesh.face_normals）
        "centers":  np.array(...),   # (m,3)   顶点平均
    }
    return patch
```

自检（单元测试级）：- 输出面片数在 m±5% 内；- 法向全部指向外（对封闭网格：(centers 附近 + 0.1·n_j) 的点在网格内部为"外法向错误"——用 trimesh 的 contains 检查或直接信任 face_normals + winding 修复）；- 面片边长直方图接近均匀（标准差/均值 < 0.5）。

7 M2 视点采样（Day 3–4）

黑盒

```
输入:  S（m 个面片）+ 传感器 FOD + 采样数 n
输出:  V = {v_1..v_n}，v_k = (位置 p_k, 视线方向 v̂_k)
例子:  v_137 = (p=(180,-120,240) mm, v̂=(-0.61,0.41,-0.67))
```

7.1 位置采样

论文原话："在工件周围欧氏空间、传感器最大 viewing range 内随机采样（冗余）"。落地做法：

```python
# src/m2_sample.py
def sample_positions(center, bbox_halfdiag, n, rng, fov_fod=(0.40, 1.00),
                     table_z, seed=0):
    # 球壳采样：方向均匀（上半球为主，排除桌面以下），
    # 距离 d ~ U(bbox_halfdiag + FOD_min + 0.05, bbox_halfdiag + FOD_max)
    # 例：halfdiag=0.30 m → d ∈ U(0.75, 1.30) m（保证视点到表面的距离落在
    #     FOD 范围内）
    # 丢弃 z < table_z 的点（相机不能钻到桌面下）
    ...
```

用固定 seed，三种 n ∈ {200,300,400} 共享同一组候选（论文明确要求三种方法共用同一候选集——保证对比公平的前提，别忘）。

7.2 视线方向：势场法（Eq.10）

$$v = \frac{\sum_i^N K \frac{d_i}{\|d_i\|^3}}{\left\| \sum_i^N K \frac{d_i}{\|d_i\|^3} \right\|}, \quad d_i = p_{vp} - p_{patch_i}$$

- 只对满足 $\left(p_{vp} - p_{patch_i}\right)^T \left(p_{vp} - p_{patch_i}\right) < d_{max}$ 的面片求和（论文未给 d_max 数值，**我们取 d_max = 0.30 m**，并用 §6.3 的诊断图调）。
- **K 实际不起作用**：Eq.10 中 K 在归一化时被约掉，只需一个常数（取 1）。

- **符号约定（重要）**：论文符号 d_i = p_vp − p_patch（面片→相机）。视线方向必须**从相机指向面片**，因此实现时取 `v̂ = normalize(Σ (p_patch_i − p_vp)/‖p_patch_i − p_vp‖³)`（即 −d_i 的加权平均）。语义：相机朝向"附近面片质量的质心方向"，$1/d^2$ 场强衰减让近处面片主导。

```
def potential_direction(p, S, d_max=0.30):
    rel = S["centers"] - p                # 面片 → 相机 (m,3)
    dist2 = (rel**2).sum(-1)              # (m,)
    mask = dist2 < d_max**2
    w = rel[mask] / dist2[mask, None]**1.5   # d/‖d‖³, 1/‖d‖² 权重
    v = w.sum(0)
    v /= np.linalg.norm(v)
    return v
```

7.3 可达性过滤（替代论文的 IK+FCL，简化偏差 #2）

```
def reachability_filter(p, base=(0.9, 0, 0), r_min=0.55, r_max=1.30, table_z=...):
    # 球形壳: r_min ≤ ‖p − base‖ ≤ r_max; z ≥ table_z + 0.1
    #  (6-DOF 臂的简化包络；不含碰撞检测——在论文中如实报告)
```

7.4 自检

- **诊断图 1**：每个视点 d_max 半径内的面片数直方图。目标：中位数 ≥ 10（太少则 d_max 太小，视线方向噪声大；太多则所有视线都指向工件中心，失去势场意义）。据此调 d_max，本复现初值 0.30 m。
- **诊断图 2**：视线方向分布应与"从外指向工件表面"一致（点积 `v̂·(center − p) > 0` 的比例应接近 100%）。
- 手算用例：一个平面（面片都在 z=0 平面），相机在 (0.5, 0, 0.2)：`v̂` 应指向平面质心附近（≈ normalize(−0.5, 0, −0.2)）。

8 M3 可见性矩阵（Day 5–7，工作量主体）

黑盒

```
输入:  V (n 视点) + S (m 面片) + 传感器参数
输出:  A ∈ {0,1}^{n×m}
例子:  A[137][347] = 1  ⇔  视点 137 能看到面片 347
```

8.1 四条判据（对每个面片的全部 3 个顶点逐一判定，全过才算可见）

设视点位置 p_c、视线方向 v̂（单位）、面片顶点 q、面片法向 n_j（向外）、面片中心 c_j。

判据 1 — FOV（30°×21°）。建相机坐标系：z_c = v̂，x_c = 任意与 v̂ 正交的单位向量（如 normalize(cross(v̂, [0,0,1]))，退化时换 [0,1,0]），y_c = cross(z_c, x_c)（左手系注意：图像坐标 y 向下也行，对称检查无影响）。对每个顶点：

```
rel = q - p_c
z = rel @ z_c; x = rel @ x_c; y = rel @ y_c
assert z > 0                          # 顶点必须在相机前方
ok_fov = (abs(x)/z <= np.tan(np.radians(15.0))) and \
         (abs(y)/z <= np.tan(np.radians(10.5)))
```

判据 2 — FOD（0.40–1.00 m）：`0.40 ≤ ‖q − p_c‖ ≤ 1.00`（每个顶点都要满足）。

判据 3 — viewing angle ≤75°：视线与表面法向的夹角：

```
los = c_j - p_c                          # 相机 → 面片中心
theta = np.arccos(np.clip(los @ n_j / np.linalg.norm(los), -1, 1))
ok_angle = theta <= np.radians(75.0)
```

判据 4 — 无遮挡：相机到每个顶点的射线不得在到达顶点前碰到其他面片：

```
import trimesh
mesh_ray = trimesh.ray.ray_triangle.RayMeshIntersector(mesh)  # 整个工件网格
for q in vertices:
    d = q - p_c
    hit = mesh_ray.intersects_location([p_c], [d])[0]
    # 排除命中点就是该顶点自身（距离 ≈ ||d|| - ε）
    ok_occ = all(np.linalg.norm(h - p_c, axis=1) > np.linalg.norm(d) - 1e-3
                 for h in hit) if len(hit) else True
```

性能提示：n×m×3 次射线查询，400×1000×3 = 120 万次。装 **embree** 加速（`pip install pyembree`，trimesh 自动启用；装不上就用默认引擎，此时把开发期规模降到 200×500，调通后再跑全量）。

8.2 矩阵组装

```
A = np.zeros((n, m), dtype=bool)
for i, vp in enumerate(V):
    for j, patch in enumerate(S):
        A[i, j] = all_four_criteria(vp, patch)
np.savez("data/outputs/A.npz", A=A)
```

8.3 覆盖率函数（fitness 与验证共用）

```
def find_coverage_ratio(A, selected_rows):
    # 覆盖 = 被选中视点任一看过的面片数 / m（注意 paper 的判据是
    # "覆盖数 ≥ δ·m"，即面片是计数单元，按面片算不按面积算）
    covered = A[selected_rows].any(axis=0)
    return covered.sum() / A.shape[1], covered
```

8.4 自检（诊断图）

- **A 的行和直方图**（每个视点能看到多少面片）：中位数应显著 >0（例如 ≥ 100 面片/视点；太小说明采样壳太远或 d_max 视线方向没对准表面）。
- **A 的列和直方图**（每个面片被多少个视点看到）：99.5% 覆盖意味着几乎每个面片至少要有几个视点能看到；**列和为 0 的面片必须为 0 个**（否则任何方法都达不到 δ=99.5%，此时增大 n 或调采样壳）。

8.5 手算单元测试（6 个，写在 tests/test_visibility.py）

#	场景	期望
1	水平面片（法向 +z），相机 (0,0,0.6)、v̂=(0,0,−1)	A=1（FOD✓ FOV✓ 角度 0°✓ 无遮挡✓）
2	同上，相机 (0,0,0.2)	A=0（FOD 下限不满足）
3	同上，相机 (0,0,0.6) 但 v̂=(1,0,0)	A=0（顶点在相机后方）
4	面片法向 +z，相机 (0.6,0,0.1) 朝面片看	A=0（角度 ≈80°>75°）

#	场景	期望
5	两个平行面片前后排列，相机只在前片正前方	后片 A=0（遮挡），前片 A=1
6	面片在 FOV 角点外 1°（v̂ 偏离 16°）	A=0（FOV 边界，验证 15°/10.5° 的实现）

9 M4 行走代价 + home pose（Day 8）

黑盒

```
输入:  V（n 视点）+ home pose
输出:  E ∈ R^{n×n}（对称，e_ij = v_i→v_j 行走时间）
       + E_home ∈ R^n（home↔各视点）
例子:  E[137][3] = 0.72 s
```

代理公式（简化偏差 #1，论文是 MoveIt/RRT+cubic spline 的关节空间时间）：

$$e(v_i, v_j) = \alpha \left\| p_i - p_j \right\| + \beta \arccos(\hat{v}_i \cdot \hat{v}_j)$$

- 取 **α = 1.2 s/m、β = 0.4 s/rad**（自选参数，标注在论文中）。理由：工业臂末端有效速度约 0.8–1.5 m/s + 姿态调整开销。
- **标定技巧**：跑通 §9 后，用 α 把 GA-SCTSP 的 cycle time 调到 **15–30 s** 区间——论文仿真 30 s 左右、真机 19.2 s，落在同数量级便于直觉对照（也便于 Phase 3 表格阅读）。
- home pose（论文只在 Fig.2 给出图像无数值，自选）：`p_h = 工件中心 + (0.9, 0, 0.6) m`，视线指向工件中心。E_home 用同一代理公式。

自检：E 对称（|E − E^T| 全零）；对角为 0；E 值分布直方图合理（0.2–3 s 量级）。

10 M5 RKGA 求解 SCTSP（Day 9—11）

10.1 问题表述（论文 Eq.8—9, 11—13，实现按此）

sequencing 表述：直接优化有序视点集 $\mathbf{x}'$，访问顺序由有序性隐含给出，TSP 约束全部消失：

$$\min_{x', n_x} \underbrace{\omega_0 n_x}_{\text{检测}} + \underbrace{\sum_{i=1}^{n_x - 1} e(x'_i, x'_{i+1}) + e(x'_h, x'_1) + e\left(x'_{n_x}, x'_h\right)}_{\text{行走（含 home 往返）}}$$

$$\text{s.t.} \qquad \text{覆盖的面片数} \geq \delta \cdot m$$

10.2 染色体编码与解码

- 染色体：长度 n 的实数向量 x ∈ [0,1]^n（random keys）。
- **decodeBySort**：`order = np.argsort(x)`——key 从小到大排序即访问顺序。
- 论文例子：keys (0.31, 0.25, 0.84, 0.64, 0.93) ⟶ 序列 2⟶1⟶4⟶3⟶5。**先写这个例子当单测。**

10.3 fitness（论文 Algo 1；实现按 Eq.11 的求和形式）

⚠️ 实现注意：论文 Algo 1 转录中第 4 步的 `e(x'_i, x'_{i+1})` 索引疑似笔误（会导致多算一条边）。**以 Eq.(11) 的 $\sum_{i=1}^{n_x - 1} e(x'_i, x'_{i+1})$ 为准**。另外论文判据是 δ' > δ（严格大于），实现用 ≥（差异可忽略，在论文中说明）。

```python
def fitness(x, A, E, E_home, w0=1.0, delta=0.995):
    order = np.argsort(x)                # decodeBySort
    covered = np.zeros(A.shape[1], bool)
    y = w0                               # 第 1 个视点的检测时间
    y += E_home[order[0]]                # home → 第 1 个视点
    covered |= A[order[0]]
    n_x = 1
    for k in range(1, len(order)):
        i = order[k]
        y += w0 + E[order[k-1], i]       # 检测 + 上一视点 → 本视点
        covered |= A[i]; n_x += 1
        if covered.sum() >= delta * A.shape[1]:
            break
    y += E_home[order[n_x-1]]            # 最后视点 → home
    return y, n_x
```

关键性质（论文的核心贡献）：**n_x 不编码在染色体里，由覆盖率动态截断——所有 offspring 自动可行，不需要约束处理。**fitness 返回 (y, n_x)，多目标比较时按 y 排序（n_x 仅作记录；若两个解 y 相同，偏好 n_x 小者）。

10.4 GA 循环（参数为自选——论文未给出，报告时注明）

```python
# 自选参数（论文未给出任何 GA 超参数）：
POP = 200              # 种群
ELITE = 0.15           # 精英比例
GENS = 150             # 代数上限（论文收敛 <100 代，150 足够）
EARLY_STOP = 30        # 最优连续不变 30 代提前停
P_CROSSOVER_BIAS = 0.7   # 均匀交叉中"取更好亲本基因"的概率
P_MUT = 0.02           # 每个基因重采样为 U(0,1) 的概率
TRIALS = 10            # 每种（方法，工件，n）跑 10 次，报 T_avg/T_std/N_vp
```

```python
def evolve(init_pop, fitness_fn, ...):
    pop = np.random.uniform(0, 1, (POP, n))
    best_history = []
    for gen in range(GENS):
        vals = np.array([fitness_fn(x)[0] for x in pop])
        order = np.argsort(vals)
        elite = pop[order[:int(POP*ELITE)]].copy()
        offspring = []
        while len(offspring) < POP - len(elite):
            a, b = tournament2(pop, vals)          # k=3 锦标赛
            better, worse = (a, b) if vals[a] < vals[b] else (b, a)
            child = np.where(rng.uniform(size=n) < P_CROSSOVER_BIAS,
                             better, worse)         # 参数化均匀交叉
            mut = rng.uniform(size=n) < P_MUT
            child[mut] = rng.uniform(size=mut.sum())  # 变异=重采样
            offspring.append(child)
        pop = np.vstack([elite, offspring])
        # 记录 best/mean，检查早停
    return best_solution
```

10.5 分阶段调试顺序（重要，别跳步）

1. 先验 **TSP** 子问题：固定一个子集（如贪心 SCP 选出的 10 个视点），染色体只排序、fitness 只算行走。验收：能收敛到稳定 tour，10 次重跑 tour 长度方差小。

2. **再验 fitness**：手算 3 个视点的染色体全排列 × 手工累加核对 y。
3. **后跑完整 SCTSP**：n=200 起步。

11 两个 baseline（Day 12–13）

三者必须共用同一组候选视点集（论文明确要求，这是公平对比的前提，也是"现象复现"的成败关键）。

11.1 Greedy-SCP-GA-TSP

```
def greedy_scp(A, delta=0.995):
    uncovered = np.ones(A.shape[1], bool); sel = []
    while uncovered.sum() / A.shape[1] > 1 - delta:    # 未达标
        gain = A[:, uncovered].sum(axis=1)             # 每个视点还能覆盖多少
        i = np.argmax(gain); sel.append(i)
        uncovered &= ~A[i]
    return sel
# 然后在固定子集上跑 RKGA-TSP（fitness 只含行走，w0·|sel| 是常数项）
```

预期特征（对照论文 Table II/III）：视点数最多（12.0/11.0/10.0）、cycle time 最差、且贪心部分确定性（论文 T_std = 0.00）。

11.2 GA-SCP-GA-TSP

```
# 自选实现（论文只说"两个独立 GA"，未给细节）：
# 二进制 GA：染色体 b ∈ {0,1}^n，fitness = w0·sum(b)
#             + PENALTY·max(0, (1-delta)·m - covered(b))
# 末代最优解做贪心修复（补足覆盖）保证可行
# 再在固定子集上跑 RKGA-TSP
```

预期特征：视点数最少（论文 9.2/9.3/9.1），但 cycle time 比 GA-SCTSP 差——**这就是要复现的核心现象**。

12 评测 harness（Day 14–15）

12.1 输出表格（对照论文 Table II/III 格式）

```
目标工件 X，方法 × 候选数(200/300/400) 网格
每个格子：T_avg / T_std / N_vp（10 trials 的均值/标准差/平均视点数）

| 方法                  | 200   | 300   | 400   |
|----------------------|-------|-------|-------|
| GA-SCP-GA-TSP        | ...   | ...   | ...   |
| Greedy-SCP-GA-TSP    | ...   | ...   | ...   |
| GA-SCTSP（本文复现）   | ...   | ...   | ...   |
```

12.2 收敛曲线（对照论文 Fig.10）

每代记录 Best / Mean fitness，画两条线。预期形态：Best 在 <100 代收敛、Mean 波动较大（论文 Fig.10 的 Best≈31.7 vs Mean≈135 这种数量级差异是正常的——种群平均值含大量不可行长序列）。

12.3 耗时记录

记录各阶段耗时（采样 / A 矩阵 / E 矩阵 / RKGA），论文瓶颈是 E 矩阵（约 3 小时 400 samples）；我们用代理公式后 E 应该毫秒级，瓶颈转移到 A 矩阵 ray casting。

13 复现正确性自检 checklist（分三级）

L1 单元级（tests/，必须全绿） - [ ] decodeBySort 的论文例子（0.31,0.25,0.84,0.64,0.93 → 2→1→4→3→5） - [ ] fitness 手算用例（3 视点全排列） - [ ] 可见性 6 个手算用例（§7.5）

L2 模式级（三个工件都需满足） - [ ] Greedy-SCP 能达到 δ=99.5%，且视点数 < 25（否则采样密度/采样壳/视线方向有问题） - [ ] GA-SCTSP cycle time < GA-SCP-GA-TSP，改善在 5–25% 区间 - [ ] GA-SCP-GA-TSP 视点数最少（"点少≠时间短"现象） - [ ] Greedy 的 T_std ≈ 0（确定性贪心） - [ ] RKGA <100–150 代收敛，10 trials 的 T_std < T_avg 的 10%

L3 数字级（可选，用于论文写作） - [ ] GA-SCTSP cycle time 经 α 标定后落在 15–30 s（论文量级） - [ ] 求解耗时 ≤ 30 min/实例

常见失败排查： - GA-SCTSP 打不过分开求解 → 先查三者候选集是否相同、E 是否对称、home 腿是否两条都算了；再查 GA-SCP 的修复步骤是否让子集"覆盖最优但空间散乱"（这正是它应该输的原因）。 - 达不到 99.5% 覆盖 → 查 A 的列和是否全 >0（§7.4）；增大 n 或放宽采样壳。 - RKGA 不收敛 → 锦标赛/精英比例/变异率微调；先跑 §9.5 的 TSP 子问题定位是编码问题还是 fitness 问题。

14 Phase 2 衔接：rollout 数据生成（Day 16—18）

用途：把 GA-SCTSP 的最优解转成 Phase 2 LoRA 微调可用的演示轨迹（研究方案 Phase 1 任务 4）。

14.1 轨迹插值（EE 空间）

```
# 相邻视点间插值：位置线性 + 姿态 slerp（用 scipy.spatial.transform.Slerp）
# 步数：每段 20 步 @ 15 Hz 语义（≈1.3 s/段，与 §8 代理时间同数量级）
# 输出每步：时间戳，EE 位姿（x,y,z,qx,qy,qz,qw），当前视点 index，累积覆盖向量 c_t
```

14.2 渲染图像

- 首选 Blender Python（paper_08 已证明可行）：headless `bpy`，加载工件网格 + 虚拟相机（内参按 paper_05 的 536×371 或统一 224×224 供 Pi0.5），沿轨迹每步渲染 RGB + 深度。
- 备选：pyrender（OpenGL offscreen）。
- **Phase 1 最小版本可只渲染视点处图像**（不渲染路径中间帧），Phase 2 需要多少再补。

14.3 覆盖向量时序（Option B/C 的条件信号）

每个 episode 输出 c_t ∈ {0,1}^m 的时序——由 A 矩阵逐视点累积得到（§7.3）。另存一份压缩摘要（覆盖率 + 未覆盖方位分箱，研究方案 §4 的翻译层输入）。

14.4 位姿扰动版本（Phase 3 R1 需要）

对每个工件：工件位姿偏移 (15 mm, 5°)（paper_01 扰动量级）× 3–5 档 → 重新跑一遍 GA-SCTSP → 输出"重规划"轨迹。这套数据是 Phase 3 里 geometric baseline 的位姿扰动对照。

14.5 输出格式

```
data/outputs/rollout/ep_001/
├── meta.json        # 工件、扰动参数、视点序列、n_x、cycle_time
├── obs_000.png ...  # 渲染 RGB（含视点帧与路径帧）
├── ee_pose_*.txt    # 每步 EE 位姿
├── coverage_*.txt   # 每步累积覆盖向量（或压缩摘要）
└── gt.json          # 真值：A 矩阵子集、覆盖时序
```

（转 LeRobot 格式是 Phase 2 的活，用 `convert_*_to_lerobot.py`，此处不提前做。）

规模验收：≥50 个 episode，其中 ≥10 个扰动版本；覆盖 3 个工件 × 200/300/400 候选的最优解。

15 参数总表（configs/base.yaml 内容）

```yaml
# ── 论文给定（勿改）──────────────────────────────
sensor:
  fov_h_deg: 30.0          # FOV 水平
  fov_v_deg: 21.0          # FOV 垂直
  fod_min: 0.40            # m
  fod_max: 1.00            # m
  max_viewing_angle_deg: 75.0
  pixels: [536, 371]       # 仅渲染用
coverage_delta: 0.995
inspection_time_w0: 1.0  # s/视点
candidates: [200, 300, 400]

# ── 复现者自选（论文未给出，报告时注明）──────────
mesh:
  target_patches: 1000
sampling:
  d_max: 0.30              # Eq.10 势场纳入距离（m）
  seed: 0
  reachability: {base: [0.9, 0, 0], r_min: 0.55, r_max: 1.30}
travel_proxy:
  alpha: 1.2               # s/m
  beta: 0.4                # s/rad
  home_pose: {offset: [0.9, 0, 0.6]}   # 相对工件中心
rkga:
  pop: 200
  elite: 0.15
  gens: 150
  early_stop: 30
  crossover_bias: 0.7
  mutation: 0.02
  trials: 10
```

16 时间线（共 2.5 周 + 3 天缓冲）

天	任务	产出/里程碑
D1	环境 + 工件准备（3 个网格就绪）	meshes/ 就绪
D1–2	M1 离散化 + 单测	patch 结构体 + 直方图自检
D3–4	M2 采样 + Eq.10 + 可达壳	V 生成 + d_max 诊断图
D5–7	M3 可见性 + 6 个单测 + embree	A.npz + 列和全 >0 验证
D8	M4 行走代价 + home pose	E.npz + 对称性检查
D9–11	M5 RKGA（TSP 子问题 → fitness → 全量）	收敛曲线
D12–13	两个 baseline	三方法同候选集可跑
D14–15	harness + 10 trials + 表格	**L2 模式级 checklist 全过**
D16–18	rollout 生成 + 扰动版本	≥50 episodes
D19–21	缓冲：调参、补测试、文档	验收达标

卡点预案：若 D7 时 ray casting 太慢 → 开发期规模降到 (200 候选, 500 面片)，D9 后补全量；若 D11 时 RKGA 不收敛 → 执行 §12 排查清单，必要时种群 500、代数 300。

17 完成后直接可用的三样东西

1. **baseline 数值表**（§11.1 格式）→ 研究方案 Phase 3 的 R1 行；
2. **rollout 数据集**（§13）→ 研究方案 Phase 2 的 LoRA 微调数据（含覆盖向量时序，直接支撑 Option B 的 V1 变体与 Option C-b 消融）；
3. **扰动版本轨迹** → Phase 3 位姿扰动对比的 geometric baseline 半边。